Autocomplete Search System

Top-5 suggestions in under 1 ms on 333 K words

Adam Jaworski · Trie & Data Ingestion
Julia Winiarz · DFS & Min-Heap Ranking

Algorithm Chain: Trie → DFS + Min-Heap

1 · Problem Statement (0:00 – 0:30)

> *As a user types into a search bar, suggest the **top 5** most likely completions in **under 50 ms**, drawing from **millions** of historical search terms ranked by popularity.*

Naïve approach — scan the full list on every keystroke

- 1 M records × 1 keystroke = 1 M comparisons → ~100 ms+
- Re-runs on **every** keystroke (5–15 keystrokes per query)
- **Unfeasible.**

Goal — push the per-keystroke cost from **O(N)** to something that does not depend on the database size.

2 · Data Input & Edge Cases (0:30 – 1:30)

Input format

- CSV with two columns: `word`, `count`
- 333 K real English unigrams (Kaggle) **or** 1.5 M synthetic records
- Live input: raw UTF-8 string from the search bar

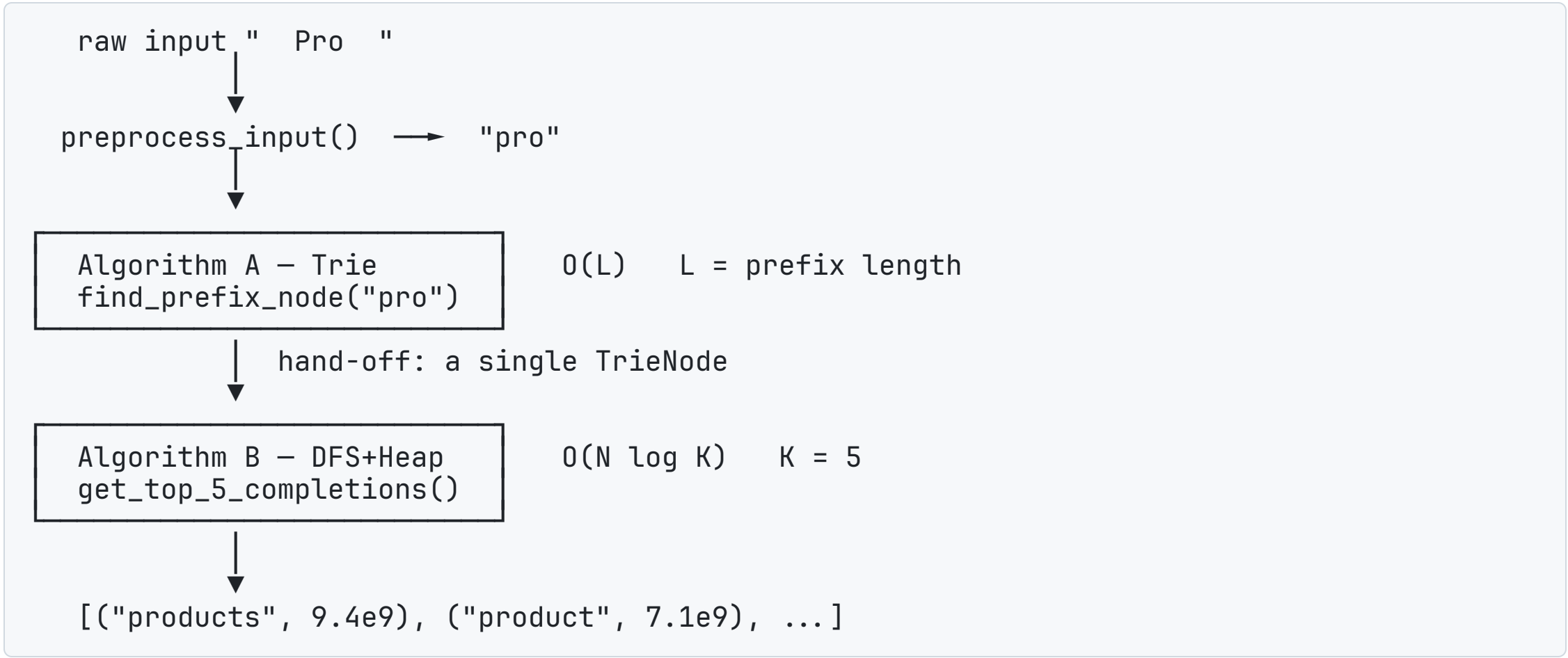
Expected output

Top 5 `(word, score)` tuples ordered by `count`, descending.

Dirty / extreme inputs we handle

Edge case	Handling
Trailing whitespace: `"  Pro  "`	`strip()` in `preprocess_input`

3 · The Algorithmic Chain (1:30 – 1:45)



Hand-off: the `TrieNode` returned by A is the *only* input to B.

4 · Algorithm A — Trie (Prefix Filtering) (1:45 – 2:30)

What it does

Walks one character at a time down a tree of hash-maps until the prefix is consumed, then returns that subtree's root.

Why a Trie

- Lookup time depends **only on prefix length** — not on database size
- Words that share a prefix share a path → no duplication
- Subtree at the end of the prefix = *all* valid completions, for free

Operation	Time	Why
`insert(word, score)`	**Θ(L)**	one node per character

5 · Algorithm B — DFS + Min-Heap (Ranking) (2:30 – 3:30)

What it does

1. DFS the subtree returned by A.
2. Visit children **best-first** (highest `max_score` first).
3. Push each end-of-word into a **Min-Heap of size 5**.
4. **Prune** any subtree whose `max_score ≤ heap[0].score`.

Why Min-Heap of size K

- Full sort = **O(N log N)**
- Heap of size K = **O(N log K)**, with K = 5 → effectively **O(N)**
- We only keep what we need

Stage	Time	Space

6 · Architectural Justification (3:30 – 4:30)

Hash-map children, not a balanced BST

	Hash Map (ours)	Balanced BST
Child lookup	**O(1)** average	O(log c)
Ordered traversal	not needed — we use `max_score` heuristic	O(log c)
Memory per node	dict overhead	tighter

We **never need ordered traversal** of children — we want *highest-score-first*, and `max_score` already encodes that. So an ordered structure would cost log time for a feature we don't use.

Min-Heap, not a sorted list

- Sorted list insertion: O(K) shifts
- Heap insertion: O(log K)

7 · Live Demo (4:30 – 5:00)

```
$ python main.py
Loading data into Trie... This might take a few seconds.
Successfully loaded 333332 words in 2.19 seconds.

> pro
Top suggestions for 'pro':
  1. products  (9.4 B)
  2. product   (7.1 B)
  3. program   (5.0 B)
  4. project   (3.8 B)
  5. profile   (3.2 B)
Search completed in 0.10 ms.

> kat
  1. kate     2. katrina    3. katie    4. kathy    5. kathleen
Search completed in 0.03 ms.

> zzzqq
No suggestions found.
Search completed in 0.001 ms.
```


≤ 1 ms per keystroke — **50× under the 50 ms budget**

8 · Verification & Take-aways

- **Correctness:** every returned word starts with the typed prefix and is sorted by descending popularity.
- **Determinism:** same input ⇒ same top-5 (no randomness, no learned weights).
- **Headroom:** 50 ms budget, 0.1 ms achieved → **500× headroom** for things we did *not* build (network, JSON, rendering, fuzzy matching).

Two algorithms, one hand-off, one cached field (`max_score`) that turns a worst-case scan into a guided search.

Q&A — backup slides next →

Adam — Trie internals, ingestion, edge cases

Julia — DFS heuristic, heap, complexity proofs

Backup A · Why O(L), not O(L · log V)?

- Each `TrieNode.children` is a Python `dict` — average **O(1)** insertion / lookup.
- `L` = length of typed prefix (≈ 5–10 characters for real searches).
- Database size **W** never appears in the lookup cost.
- Therefore: doubling the dataset **does not slow down a query.**

Backup B · Why store `max_score` at every node?

```
# in TrieNode
self.max_score = 0   # max popularity of any word in this subtree

# in Trie.insert(word, score)
node.max_score = max(node.max_score, score)   # propagated down the path
```

Effect in DFS:

```
if len(top_5_heap) == 5 and node.max_score ⩽ top_5_heap[0][0]:
    return   # whole subtree skipped
```

Cost: one extra integer per node + one `max()` per insert.

Benefit: prefix `""` (entire 333 K-word trie) finishes in 0.09 ms.

Backup C · When does this design break?

Scenario	Effect	Mitigation
Billions of records	RAM exhaustion (`O(W·L)`)	Radix Tree / DAWG
Unicode / emoji search	dict still O(1), just bigger keys	already works
Fuzzy / typo tolerance	not supported by pure Trie	add Levenshtein-automaton on top
Personalized ranking	popularity is global	weight `score` by user signal at query time